
Introduction

POSIX threads (pthreads) are a standardized interface on operating system threads.

Compiling a pthreads Program

Relevant headers aside (they are discussed below), a program wishing to use pthreads must link against the pthreads library. Here is an example invocation of gcc demonstrating this:

```
gcc -pedantic -Wall -o theaded_program src.c -lpthread
```

The -l flag specifies the name of a library to link against (pthread, in our case); since pthreads is a system library, gcc knows where to find it.

Creating Threads

Any program using pthreads will need to include pthread.h. Below is the Hello World of pthreads programs:

```
#include <pthread.h>
#include <stdio.h>

void * entry_point(void *arg)
{
    printf("Hello world!\n");

    return NULL;
}

int main(int argc, char **argv)
{
    pthread_t thr;
    if(pthread_create(&thr, NULL, &entry_point, NULL))
    {
        printf("Could not create thread\n");
        return -1;
    }

    if(pthread_join(thr, NULL))
    {
        printf("Could not join thread\n");
        return -1;
    }
    return 0;
}
```

There are two functions of note here:

- **pthread_create**: Spawns a new pthread using the given function as an *entry point*. This entry point can be considered the "main" function of that thread of execution.
- **pthread_join**: Causes the calling thread to wait until the given thread terminates.

Another point worth noting is that the thread entry point takes a single argument; the last argument to `pthread_create` is the argument that is passed to the entry point.

Similarly, threads can *return* a value to the function that calls `pthread_join` on them. See the man page for details.

Exiting a Thread

In the above example, `entry_point` returns a `NULL` pointer to the thread that calls `pthread_join` on it. Sometimes, it may be desirable to exit a thread from some function other than the entry point.

```
#include <pthread.h>
#include <stdio.h>

void other_function()
{
    pthread_exit(NULL);
}

void * entry_point(void *arg)
{
    printf("Hello world!\n");
    other_function();
    printf("Hello again?\n");
    return NULL;
}

int main(int argc, char **argv)
{
    pthread_t thr;
    if(pthread_create(&thr, NULL, &entry_point, NULL))
    {
        printf("Could not create thread\n");
        return -1;
    }

    if(pthread_join(thr, NULL))
    {
        printf("Could not join thread\n");
        return -1;
    }
    return 0;
}
```

In this example, note that "Hello again?" is never printed. The thread exits in `other_function` and `entry_point` never returns. The argument to `pthread_exit` is a value to be returned to the joining thread.

Synchronization

The pthreads specification provides many synchronization primitives; we will cover three in this primer:

- Barriers
- Mutexes
- Semaphores

Barriers

Some parallel computations need to "meet up" at certain points before continuing. This can, of course, be accomplished with semaphores, but another construct is often more convenient: the barrier (the pthreads library `pthread_barrier_t`). As a motivating example, take this program:

```

#define _XOPEN_SOURCE 600

#include <pthread.h>
#include <stdlib.h>
#include <stdio.h>

#define ROWS 10000
#define COLS 10000
#define THREADS 10

double initial_matrix[ROWS][COLS];
double final_matrix[ROWS][COLS];
// Barrier variable
pthread_barrier_t barr;

extern void DotProduct(int row, int col,
                      double source[ROWS][COLS],
                      double destination[ROWS][COLS]);
extern double determinant(double matrix[ROWS][COLS]);

void * entry_point(void *arg)
{
    int rank = (int)arg;
    for(int row = rank * ROWS / THREADS; row < (rank + 1) * THREADS; ++row)
        for(int col = 0; col < COLS; ++col)
            DotProduct(row, col, initial_matrix, final_matrix);

    // Synchronization point
    int rc = pthread_barrier_wait(&barr);
    if(rc != 0 && rc != PTHREAD_BARRIER_SERIAL_THREAD)
    {
        printf("Could not wait on barrier\n");
        exit(-1);
    }

    for(int row = rank * ROWS / THREADS; row < (rank + 1) * THREADS; ++row)
        for(int col = 0; col < COLS; ++col)
            DotProduct(row, col, final_matrix, initial_matrix);
}

int main(int argc, char **argv)
{
    pthread_t thr[THREADS];

    // Barrier initialization
    if(pthread_barrier_init(&barr, NULL, THREADS))
    {
        printf("Could not create a barrier\n");
        return -1;
    }

    for(int i = 0; i < THREADS; ++i)
    {
        if(pthread_create(&thr[i], NULL, &entry_point, (void*)i))
        {
            printf("Could not create thread %d\n", i);
            return -1;
        }
    }

    for(int i = 0; i < THREADS; ++i)
    {
        if(pthread_join(thr[i], NULL))
        {
            printf("Could not join thread %d\n", i);
            return -1;
        }
    }
}

```

```

    double det = Determinant(initial_matrix);
    printf("The determinant of M^4 = %f\n", det);

    return 0;
}

```

This program spawns a number of threads, assigning each to compute *part* of a matrix multiplication. Each thread then uses the result of that computation in the next phase: another matrix multiplication.

There are a few things to note here:

1. The barrier declaration at the top
2. The barrier initialization in `main`
3. The point where each thread *waits* for its peers to finish.

NOTE

The preprocessor definition of `_XOPEN_SOURCE` at the top of the program is important; without it, the barrier prototypes are not defined in `pthread.h`. The definition must come *before* any headers are included.

Mutexes

The pthreads library provides a basic synchronization primitive: `pthread_mutex_t`. The declarations required to use pthreads mutexes are included in `pthread.h`. This is a standard mutex with lock and unlock operations; see this example:

```

#include <pthread.h>
#include <stdio.h>
#include <math.h>

#define ITERATIONS 10000

// A shared mutex
pthread_mutex_t mutex;
double target;

void* opponent(void *arg)
{
    for(int i = 0; i < ITERATIONS; ++i)
    {
        // Lock the mutex
        pthread_mutex_lock(&mutex);
        target -= target * 2 + tan(target);
        // Unlock the mutex
        pthread_mutex_unlock(&mutex);
    }

    return NULL;
}

int main(int argc, char **argv)
{
    pthread_t other;

    target = 5.0;

    // Initialize the mutex
    if(pthread_mutex_init(&mutex, NULL))
    {
        printf("Unable to initialize a mutex\n");
        return -1;
    }
}

```

```

    if(pthread_create(&other, NULL, &opponent, NULL))
    {
        printf("Unable to spawn thread\n");
        return -1;
    }

    for(int i = 0; i < ITERATIONS; ++i)
    {
        pthread_mutex_lock(&mutex);
        target += target * 2 + tan(target);
        pthread_mutex_unlock(&mutex);
    }

    if(pthread_join(other, NULL))
    {
        printf("Could not join thread\n");
        return -1;
    }

    // Clean up the mutex
    pthread_mutex_destroy(&mutex);

    printf("Result: %f\n", target);

    return 0;
}

```

The important functions for managing mutexes are:

- `pthread_mutex_init`: Initialize a new mutex.
- `pthread_mutex_destroy`: Clean up a mutex that is no longer needed.
- `pthread_mutex_lock`: Acquire a mutex (blocking if it is not available).
- `pthread_mutex_unlock`: Release a mutex that you previously locked.

Semaphores

The pthreads library itself does not provide a semaphore; however, a separate POSIX standard does define them. The necessary declarations to use these semaphores are contained in `semaphore.h`.

NOTE: Do not confuse these with SystemV semaphores which are in `sys/sem.h`.

```

#include <semaphore.h>
#include <pthread.h>
#include <stdio.h>

#define THREADS 20

sem_t OKToBuyMilk;
int milkAvailable;

void* buyer(void *arg)
{
    // P()
    sem_wait(&OKToBuyMilk);
    if(!milkAvailable)
    {
        // Buy some milk
        ++milkAvailable;
    }
    // V()
    sem_post(&OKToBuyMilk);

    return NULL;
}

```

```

int main(int argc, char **argv)
{
    pthread_t threads[THREADS];

    milkAvailable = 0;

    // Initialize the semaphore with a value of 1.
    // Note the second argument: passing zero denotes
    // that the semaphore is shared between threads (and
    // not processes).
    if(sem_init(&OKToBuyMilk, 0, 1))
    {
        printf("Could not initialize a semaphore\n");
        return -1;
    }

    for(int i = 0; i < THREADS; ++i)
    {
        if(pthread_create(&threads[i], NULL, &buyer, NULL))
        {
            printf("Could not create thread %d\n", i);
            return -1;
        }
    }

    for(int i = 0; i < THREADS; ++i)
    {
        if(pthread_join(threads[i], NULL))
        {
            printf("Could not join thread %d\n", i);
            return -1;
        }
    }

    sem_destroy(&OKToBuyMilk);

    // Make sure we don't have too much milk.
    printf("Total milk: %d\n", milkAvailable);

    return 0;
}

```

The semaphore API has several functions of note:

- **sem_init**: Initialize a new semaphore. Note, the second argument denotes *how* the semaphore will be shared. Passing zero denotes that it will be shared among **threads** rather than processes. The final argument is the initial value of the semaphore.
- **sem_destroy**: Deallocate an existing semaphore.
- **sem_wait**: This is the P() operation.
- **sem_post**: This is the V() operation.

Relevant Man Pages

Man pages for all of the necessary library functions should be available on every CSL Linux system:

	Basic Management	Barriers	Mutexes	Semaphores
Creation	pthread_create	pthread_barrier_init	pthread_mutex_init	sem_init
Destroy	pthread_exit	pthread_barrier_destroy	pthread_mutex_destroy	sem_destroy
Waiting	pthread_join	pthread_barrier_wait	-	-
Acquisition	-	-	pthread_mutex_lock	sem_wait
Release	-	-	pthread_mutex_unlock	sem_post

Resources

Below are some additional resources:

- [Lawrence Livermore National Labs Pthreads Tutorial](#)
- [IBM developerWorks POSIX Threads explained \(I\)](#)
- [IBM developerWorks POSIX Threads explained \(II\)](#)
- [IBM developerWorks POSIX Threads explained \(III\)](#)